

Running IFN (login-only) on Windows, step by step

This is the slow, no-assumptions version. If you have never touched Docker or a terminal, you can still follow this. Every command is copy-paste. Lines that start with # are comments, you do not type them.

You are going to:

1. Install three programs (Docker Desktop, Git, Node.js).
2. Get the code.
3. Start the backend (the Docker stack).
4. Load the database schema.
5. Start the frontend.
6. Make yourself an admin and log in.

Plan for about 30 to 45 minutes the first time, most of it waiting on downloads.

0. What you need

- Windows 10 or 11, 64-bit, with admin rights (you can install software).
 - About 8 GB free disk space.
 - An internet connection.
-

1. Install the three programs

1a. Docker Desktop (this runs the backend)

1. Go to <https://www.docker.com/products/docker-desktop> and download Docker Desktop for Windows.
2. Run the installer. When it asks, leave **“Use WSL 2 instead of Hyper-V”** checked. WSL2 is the Linux engine Docker uses under the hood, and our scripts need it.
3. Restart the PC if it asks.
4. Open Docker Desktop from the Start menu. Wait until the little whale icon in the system tray stops animating and says “Docker Desktop is running”.
5. If it complains that WSL needs updating, open PowerShell as Administrator (right-click PowerShell, Run as administrator) and run:

```
wsl --update
```

Then reopen Docker Desktop.

You will know it works when you can open a terminal (see step 1d) and run docker version without an error.

1b. Git (this downloads the code, and gives you a Linux-style shell)

1. Go to <https://git-scm.com/download/win> and download “64-bit Git for Windows Setup”.
2. Run the installer. Click Next on every screen, the defaults are fine.
3. This installs **Git Bash**, a terminal that understands the .sh scripts in this project. We use Git Bash for everything below.

1c. Node.js (this runs the frontend)

1. Go to <https://nodejs.org> and download the **LTS** version for Windows.
2. Run the installer, defaults are fine.
3. Check it worked: open Git Bash (step 1d) and run `node --version`. You should see something like v20.x or higher.

1d. Opening a terminal (Git Bash)

Whenever this guide says “in a terminal”, open **Git Bash**: press the Windows key, type Git Bash, press Enter. A black window opens. That is where you type commands.

To paste into Git Bash, right-click and choose Paste (Ctrl+V usually works too).

2. Get the code

In Git Bash:

```
# go to your home folder  
cd ~
```

```
# download the project (replace the URL with your repo)  
git clone https://github.com/YOUR-NAME/ifn-login-only.git
```

```
# go into it  
cd ifn-login-only
```

From now on, every command assumes you are inside this ifn-login-only folder unless it says otherwise.

3. Set up the backend config

The backend reads its settings from a file called `selfhost/.env`. That file is not in the repo (it can hold secrets), so you create it from the template:

```
cp selfhost/.env.example selfhost/.env
```

That is it for local use. The template already has working values: the captcha uses Cloudflare's always-pass test keys, the ports are set, and the database password is a demo one. **Do not deploy this to the internet as-is.** See docs/settings-and-debugging.md and docs/email-smtp.md for what to change before going live.

4. Start the backend (the Docker stack)

Make sure Docker Desktop is running (whale icon steady). Then:

```
cd selfhost
docker compose up -d
```

- docker compose is the tool that starts all the containers.
- up means start them.
- -d means “detached”, so they run in the background and give you your terminal back.

The first time, this downloads several gigabytes of images. Be patient. When it finishes, check they are healthy:

```
docker compose ps
```

You want to see a list of containers with status Up and (healthy). The important ones are kong, auth, db, storage, and mailpit.

5. Load the database schema

A fresh database is empty. This script creates all the tables, security rules, and functions:

```
# still inside the selfhost folder
./apply-schema.sh
```

This runs the .sh script through Git Bash. It waits for the storage service to finish setting up, then applies every SQL file twice (the second pass cleans up ordering). When it is done you should see a column of green check marks and no red × lines. If you see a ×, see the troubleshooting section in docs/settings-and-debugging.md.

Now deploy the edge functions (small server programs like “create member”):

```
# copies the function code into the place the runtime reads from, then restarts it
cp -r ../supabase/functions/. volumes/functions/
docker compose up -d functions
```

6. Start the frontend

Open a **second** Git Bash window (keep the backend one open). Then:

```
cd ~/ifn-login-only/web
```

```
# install the frontend's libraries (first time only, takes a few minutes)
npm install
```

```
# start the dev server, pointed at your local backend
npm run dev:local
```

Leave this running. It prints a line like Local: <http://localhost:5173/>. Open that in your browser.

Why dev:local and not dev? npm run dev points at the live cloud database. npm run dev:local points at the backend you just started on this machine. Always use dev:local here. More on this in docs/frontend-backend.md.

7. Make yourself an admin and log in

Sign-up is off (admins create accounts), so the very first admin is made by hand.

1. Open the admin dashboard (Studio) in your browser: **<http://localhost:8010>**
 - o If it asks for a username and password, they are in selfhost/.env:
DASHBOARD_USERNAME and DASHBOARD_PASSWORD.
2. In Studio, click **Authentication** in the left sidebar, then **Add user**. Enter your email and a password, and tick **Auto Confirm User** so you can log in right away. Click Save. Copy the new user's ID (a long string like a1b2c3...).
3. Click **SQL Editor** in the left sidebar, paste this (replace the id), and Run:

```
update public.profiles
set role = 'admin', onboarded = true, must_change_password = false
where id = 'PASTE-THE-USER-ID-HERE';
```

4. Go to the app at **<http://localhost:5173/login>** and log in with that email and password.

You are in. To add more people, go to the **Admin Panel** in the app, **Add member** tab. Each person gets a temporary password and is forced to set their own on first login. See the app's Add-member screen, it explains the welcome and decline email buttons.

Do the .sh files work on Windows?

Yes, as long as you run them in **Git Bash** (or inside WSL), which you installed in step 1b. They will NOT work if you double-click them or run them in the old Command Prompt / PowerShell, because those do not understand bash. Always run ./apply-schema.sh and friends from a Git Bash window.

The other scripts live in selfhost/utils/ (key generation, password change). Same rule: run them from Git Bash.

Starting and stopping later

`cd ~/ifn-login-only/selfhost`

```
docker compose stop    # pause everything, keeps your data
docker compose up -d   # start it again
docker compose ps      # see what is running
```

Never run `docker compose down -v`. The `-v` deletes the database volume and wipes all your data. Plain stop and down are safe.

To stop the frontend, go to its Git Bash window and press Ctrl+C.

Quick reference: where things live

Thing	Address
The app (frontend)	<code>http://localhost:5173</code>
Admin dashboard (Studio)	<code>http://localhost:8010</code>
Email inbox (Mailpit, local mail)	<code>http://localhost:8035</code>
The API gateway (Kong)	<code>http://localhost:8010</code>
Database (for tools like DBeaver)	<code>localhost:5443</code> , user <code>postgres</code>

Next reading: `docs/env-reference.md` (what every setting does) and `docs/settings-and-debugging.md` (what to change and how to fix problems).